

Pandas: Data Cleaning and Preparation

PTP Program



Training Guidelines

Persyaratan dan Ekspektasi untuk Peserta Pelatihan:

1. **Prerequisite:** Peserta diharapkan sudah memiliki pengetahuan dasar tentang python, mulai dari tipe data hingga penggunaan package.
2. **Active Participation:** Peserta harus terlibat aktif dengan materi pelatihan, mencatat poin-poin penting dan penjelasan yang diberikan oleh instruktur.
3. **Clear Instruction:** Instruktur bertanggung jawab untuk menyampaikan materi pelatihan dengan cara yang jelas dan ringkas, memastikan semua peserta memahami topik yang dibahas.
4. **Hands-on Practice:** Sebagian besar pelatihan akan melibatkan latihan praktis yang dilakukan menggunakan **Google Colab (Jupyter Notebook)**. Anda dapat mengakses Google Colab yang tersedia di slide terakhir.

List Of Contents

- Introduction to Pandas
- Basic Statistics
- Exploring Dataset
- Practice

Introduction to Pandas

Apa itu Pandas?

Pandas adalah tools analisis dan manipulasi data open source yang cepat, powerful, fleksibel, dan mudah digunakan, dibangun di atas bahasa pemrograman Python.

Berikut Kelebihan dari Pandas:

- **Struktur data yang efisien:** Pandas menyediakan dua struktur data utama, yaitu Series (satu dimensi) dan DataFrame (dua dimensi), yang sangat cocok untuk mewakili berbagai jenis data.
- **Fitur yang lengkap:** Pandas menawarkan berbagai fungsi untuk melakukan manipulasi data, seperti:
 - Membaca dan menulis data dari berbagai format file (CSV, Excel, SQL, dll.)
 - Memilih data, mengubah, dan menghapus data
 - Menggabungkan dan menggabungkan DataFrame
 - Menghitung statistik deskriptif
 - Membuat pivot table
- **Ekosistem yang kuat:** Pandas terintegrasi dengan baik dengan library Python lainnya, seperti NumPy, Matplotlib, dan Scikit-learn, sehingga memungkinkan Anda melakukan analisis data yang lebih kompleks.

Introduction to Pandas

Manfaat Pandas

Berikut ini adalah kegunaan Pandas dalam Analisis Data:

- **Pembersihan data:** Menghapus data yang duplikat, mengisi data yang hilang, dan mengubah format data.
- **Eksplorasi data:** Mencari pola, tren, dan anomali dalam data.
- **Transformasi data:** Mengubah data ke dalam format yang sesuai untuk analisis lebih lanjut.
- **Visualisasi data:** Membuat grafik dan plot untuk memvisualisasikan data dengan lebih mudah karena terintegrasi dengan matplotlib.
- **Analisis statistik:** Menghitung statistik deskriptif dan melakukan uji statistik.

Introduction to Pandas

Series & DataFrame

Pandas menyediakan dua struktur data utama, yaitu Series dan DataFrame, yang sangat berguna untuk memanipulasi dan menganalisis data.

Series adalah struktur data satu dimensi yang mirip dengan array NumPy, tetapi dengan tambahan fitur seperti label (indeks) untuk setiap elemen. Series dapat berisi berbagai tipe data, mulai dari angka, teks, hingga objek Python lainnya.

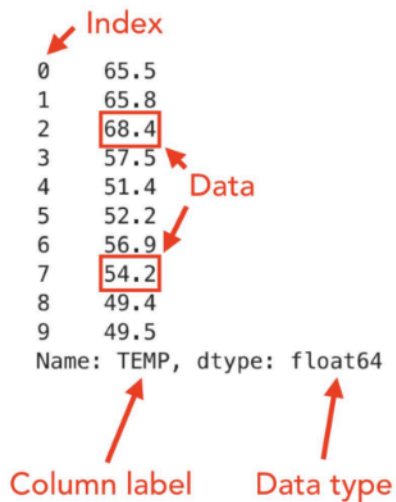
DataFrame adalah struktur data dua dimensi yang mirip dengan tabel dalam spreadsheet. DataFrame terdiri dari beberapa kolom, dan setiap kolom adalah sebuah Series.

Series		Series		DataFrame	
Website		Visited		Website	Visited
0	datagy.io	0	2023-01-23	0	datagy.io 2023-01-23
1	google.com	1	2023-01-24	1	google.com 2023-01-24
2	bing.com	2	2023-01-04	2	bing.com 2023-01-04

Introduction to Pandas

Series Anatomy

Series dikenal sebagai kolom tabel yang terdiri dari nilai, indeks baris, nama kolom, dan tipe data.



The diagram shows a Pandas Series with 10 elements. Red arrows point to specific parts: 'Index' points to the row numbers 0-9; 'Data' points to the values; 'Column label' points to 'TEMP'; and 'Data type' points to 'float64'. Two values, 68.4 and 54.2, are highlighted with red boxes.

0	65.5
1	65.8
2	68.4
3	57.5
4	51.4
5	52.2
6	56.9
7	54.2
8	49.4
9	49.5

Name: TEMP, dtype: float64

Column label Data type

```
import pandas as pd

pd.Series([65.5, 65.8, 68.4,
           57.5, 51.4, 52.2,
           56.9, 54.2, 49.4, 49.5])
```

Introduction to Pandas

DataFrame Anatomy

DataFrame adalah tabel yang terdiri dari label indeks, nama kolom, dan nilai.

	YEARMODA	TEMP	MAX	MIN
0	20160601	65.5	73.6	54.7
1	20160602	65.8	80.8	55.0
2	20160603	68.4	77.9	55.6
3	20160604	57.5	70.9	47.3
4	20160605	51.4	58.3	43.2
5	20160606	52.2	59.7	42.8
6	20160607	56.9	65.1	45.9
7	20160608	54.2	60.4	47.5
8	20160609	49.4	54.1	45.7
9	20160610	49.5	55.9	43.0

```
import pandas as pd

pd.DataFrame({
    'Name': ['John', 'Emma', 'Mike', 'Lisa'],
    'Age': [25, 28, 31, 27],
    'City': ['New York', 'London', 'Sydney', 'Paris']
})
```


Introduction to Pandas

Data Loading

Untuk memuat data ke dalam pandas DataFrame dari suatu sumber, Anda dapat menggunakan berbagai metode tergantung pada sumber datanya. Berikut ini beberapa cara umum untuk memuat data di pandas:

```
import pandas as pd

df = pd.read_csv('filename.csv', delimiter=',')

df = pd.read_excel('filename.xlsx', sheet_name='Sheet1')
```

Ingatlah bahwa `pd.read_csv` juga dapat membaca ekstensi lain seperti `.tsv`, `.txt`, tetapi Anda hanya perlu menentukan pemisah data. Kita juga dapat menentukan url yang mengunduh atau membaca data secara langsung:

```
df = pd.read_csv('https://url.com/filename.csv')
```



Basic Statistics

Introduction to Pandas

Summary Statistics

Di pandas, Anda dapat melakukan operasi agregasi untuk menghitung Summary Statistics atau menerapkan fungsi “groupby”. Berikut ini beberapa operasi agregasi umum di pandas:

```
df.mean() # Compute the mean value of each numeric column
df.sum() # Compute the sum of each numeric column
df.count() # Count the number of non-null values in each column
df.min() # Compute the minimum value in each column
df.max() # Compute the maximum value in each column
df.median() # Compute the median value of each numeric column
df.std() # Compute the standard deviation of each numeric column
df.var() # Compute the variance of each numeric column
df.quantile(q) # Compute the specified quantile (0 to 1) for each numeric column
df.agg({'column1': 'sum', 'column2': 'mean'}) # Computes the sum of 'column1' and the mean of 'column2'
```

Introduction to Pandas

Summary Statistics

Contoh Group By untuk satu kolom:

```
df.groupby('column_name').mean() #Single aggregation  
df.groupby('column_name').agg(['mean', 'sum', 'count']) #Multi aggregation
```

Contoh Group By untuk dua kolom:

```
df.groupby(['column1', 'column2']).mean() #Single aggregation  
df.groupby(['column1', 'column2']).agg(['mean', 'sum', 'count']) #Multi aggregation
```



Exploring Data

Exploring Data

Exploring Data - Basic Methods

Pandas sangat berguna untuk mengeksplorasi data kita. Mari kita lihat apa saja yang bisa kita lakukan dengan data yang tersimpan dalam variable DataFrame `df`.

- `df.head()` - Metode ini mengembalikan baris `n` pertama dari DataFrame. Secara default, ini akan mengembalikan 5 baris pertama.
- `df.tail()` - Metode ini mengembalikan baris `n` terakhir dari DataFrame. Secara default, ini akan mengembalikan 5 baris terakhir.
- `df.shape` - Atribut ini mengembalikan tupel yang mewakili dimensi dari DataFrame (jumlah baris, jumlah kolom).
- `df.info()` - Metode ini memberikan ringkasan singkat dari DataFrame, termasuk jumlah nilai non-null di setiap kolom dan tipe data dari kolom tersebut.
- `df.describe()` - Metode ini menghasilkan statistik deskriptif dari kolom numerik dalam DataFrame, termasuk jumlah, rata-rata, standar deviasi, minimum, maksimum, dan nilai kuartil.

Exploring Data

Exploring Data - Basic Methods

- `df.columns`: Atribut ini mengembalikan label-label kolom dari DataFrame.
- `df.index`: Atribut ini mengembalikan label-label indeks dari DataFrame.
- `df.dtypes`: Atribut ini mengembalikan tipe data dari setiap kolom dalam DataFrame.
- `df.isnull()`: Metode ini mengembalikan DataFrame dengan bentuk yang sama dengan DataFrame asli, di mana setiap elemen adalah nilai boolean yang menunjukkan apakah elemen yang bersesuaian hilang (NaN) atau tidak.
- `df.nunique()`: Metode ini mengembalikan jumlah nilai unik dalam setiap kolom dari DataFrame.
- `df.value_counts()`: Metode ini mengembalikan frekuensi kemunculan nilai unik dalam sebuah kolom dari DataFrame.



Basic Data Manipulation

Slicing - Accessing Columns and Rows

Mengakses Kolom:

- **Notasi Kurung Siku (Bracket Notation):** Anda dapat menggunakan tanda kurung siku [] dengan nama kolom di dalamnya untuk mengakses satu kolom atau beberapa kolom.
 - `df['column_name']`: Mengakses satu kolom sebagai Series.
 - `df[['column1', 'column2']]`: Mengakses beberapa kolom sebagai DataFrame.
- **Notasi Titik (Dot Notation):** Jika nama kolom adalah identifier Python yang valid dan tidak bertabrakan dengan atribut DataFrame, Anda dapat menggunakan notasi titik untuk mengakses kolom.
 - `df.column_name`: Mengakses satu kolom sebagai Series.

Mengakses Baris:

- **.iloc[]:** Anda dapat menggunakan `.iloc[]` dengan indeks berbasis integer untuk mengakses baris berdasarkan posisinya (indeks).
 - `df.iloc[row_index]`: Mengakses satu baris sebagai Series.
 - `df.iloc[start_index:end_index]`: Mengakses rentang baris sebagai DataFrame.
- **.loc[]:** Anda dapat menggunakan `.loc[]` dengan indeks berbasis label untuk mengakses baris berdasarkan label indeksnya atau kondisi boolean.
 - `df.loc[row_label]`: Mengakses satu baris sebagai Series.
 - `df.loc[start_label:end_label]`: Mengakses rentang baris sebagai DataFrame.
 - `df.loc[condition]`: Mengakses baris berdasarkan kondisi boolean.

Basic Data Manipulation

Query/Filtering

Di pandas, Anda dapat melakukan kueri atau operasi pemfilteran untuk memilih baris tertentu atau memfilter data berdasarkan kondisi tertentu. Berikut ini beberapa teknik untuk membuat kueri atau memfilter di pandas:

1. Boolean Indexing:
 - a. Menggunakan satu kondisi

```
df[df['column_name'] > 5] # Selects rows where the values  
in 'column_name' are greater than 5
```

- b. Menggunakan beberapa kondisi

```
df[(df['column1'] > 5) & (df['column2'] == 'value')] # Selects rows  
where 'column1' is greater than 5 and 'column2' equals 'value'
```

Anda dapat menggabungkan kondisi menggunakan operator logika seperti **&** (and), **|** (or), dan **~** (not)

Basic Data Manipulation

Query/Filtering

2. query method:

```
df.query('column_name > 5') # Selects rows where the values in  
'column_name' are greater than 5
```

3. isin method:

```
df[df['column_name'].isin(['value1', 'value2', 'value3'])] #  
Selects rows where 'column_name' is in the specified list of value
```

4. str method (for string columns):

```
df[df['column_name'].str.contains('substring')] # Selects rows  
where 'column_name' contains a specific substring
```

Basic Data Manipulation

Query/Filtering

5. loc method (label-based indexing):

```
df.loc[df['column_name'] > 5, 'column2'] # Selects values in 'column2'
for rows where the values in 'column_name' are greater than
```

6. query syntax:

```
df.query('column1 > 5 and column2 == "value"') # Selects rows based on
a complex query using column conditions
```



External References (Practice)

Data exploration
practice

Visit Here